Rust Programming Language

A high level overview of the *latest* highly admired programming language by developers.

Feedback

Authors reread their sentences are they write them.

Looking for a yellow car - they are out there but you won't notice them until you start looking.

Foreword

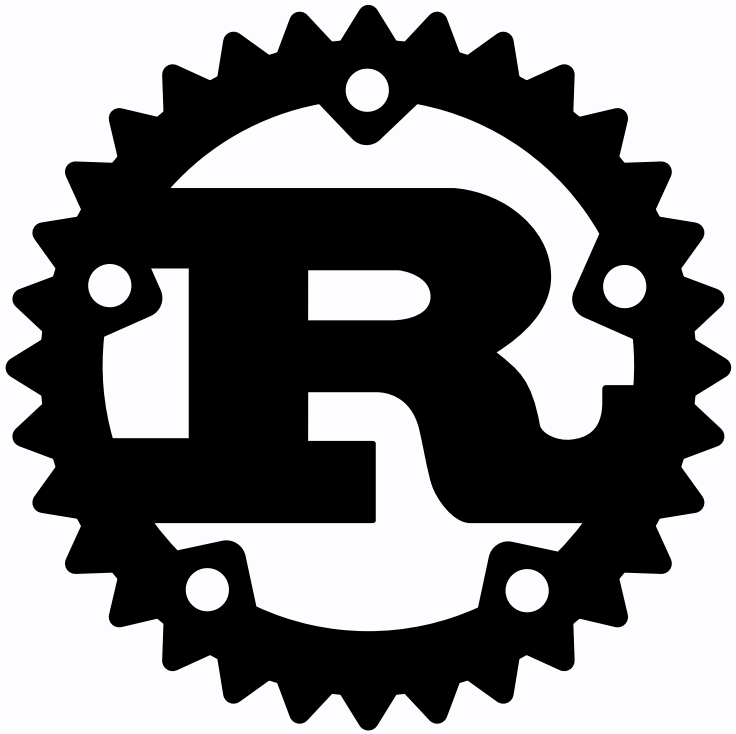
📝 Contribute on GitHub

🔗 View Online neave.dev/rust

⬇️ Download PowerPoint and PDF

What is Rust?

Based of the Rust Fungus. The logo is a bike ring because most of the developers who started Rust like to ride bikes



Why Rust?

Blazing Fast ⚡

> Don't count your servers, make your servers count.

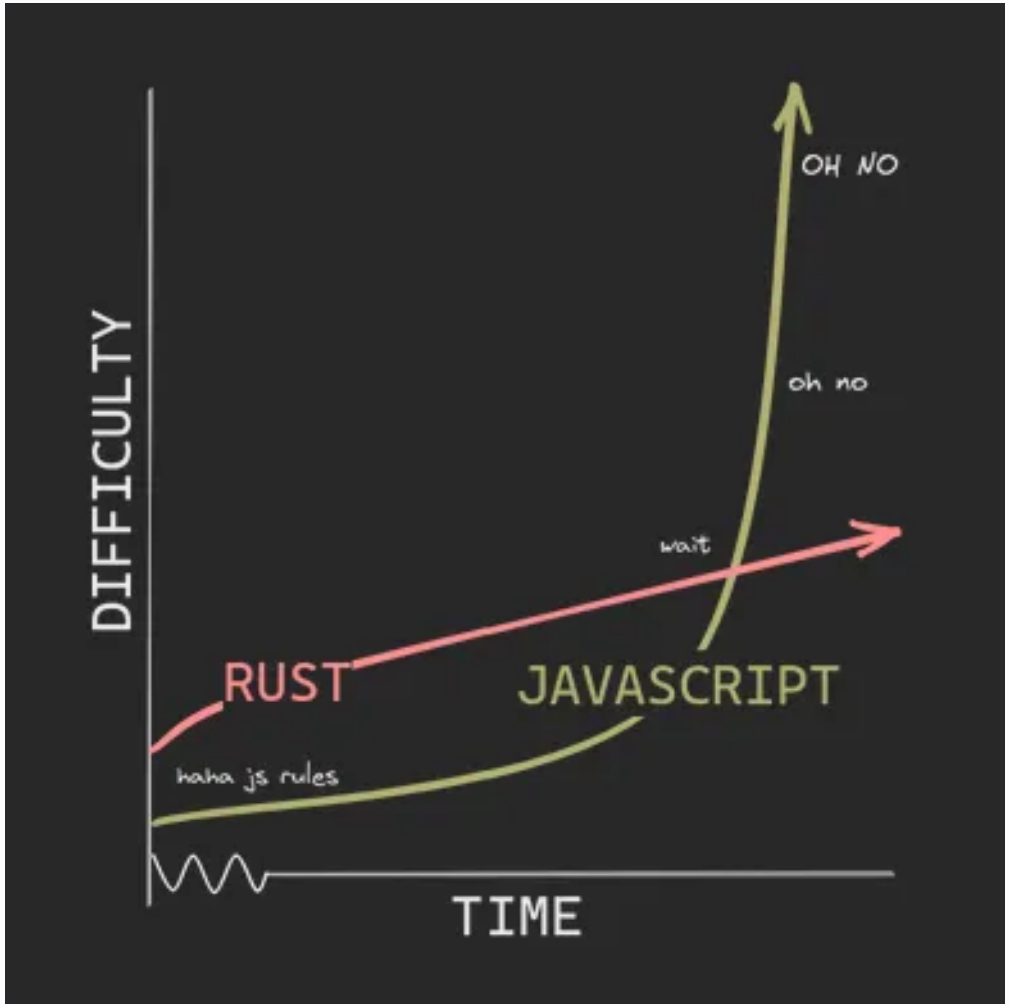
Make Cost a Non-Functional Requirement

> Ruby & Python are more than 50x more expensive than C++ and Rust

The Frugal Architect

(quoted in Why Rust? - Serverless Rust)

Javascript vs Rust



Rust in the wild

- Rust is in the Linux
- Microsoft seeks Rust developers to rewrite core C# code
- Mozilla re-focuses on Web Engine Rust based 'Servo' in 2024.
- CloudFlare open sources their http proxy (alternative to NGINX) called Pingora
- RedoxOs
- Warp Terminal - an AI Powered Terminal
- Using WebAssembly for Extension Development

- The White House Office of the National Cyber Director (ONCD) called on developers to reduce the risk of cyberattacks by using programming languages that don't have memory safety vulnerabilities.
- NSA cybersecurity information sheet recommends we use memory safe languages C#, Go, Java, Ruby, Swift and *Rust*.

- Rust developers at Google are twice as productive as C++ teams

> "So we see reduced memory usage in the services that we've moved from Go ... and we see a decreased defect rate over time in those services that have been rewritten in Rust – so increasing correctness."

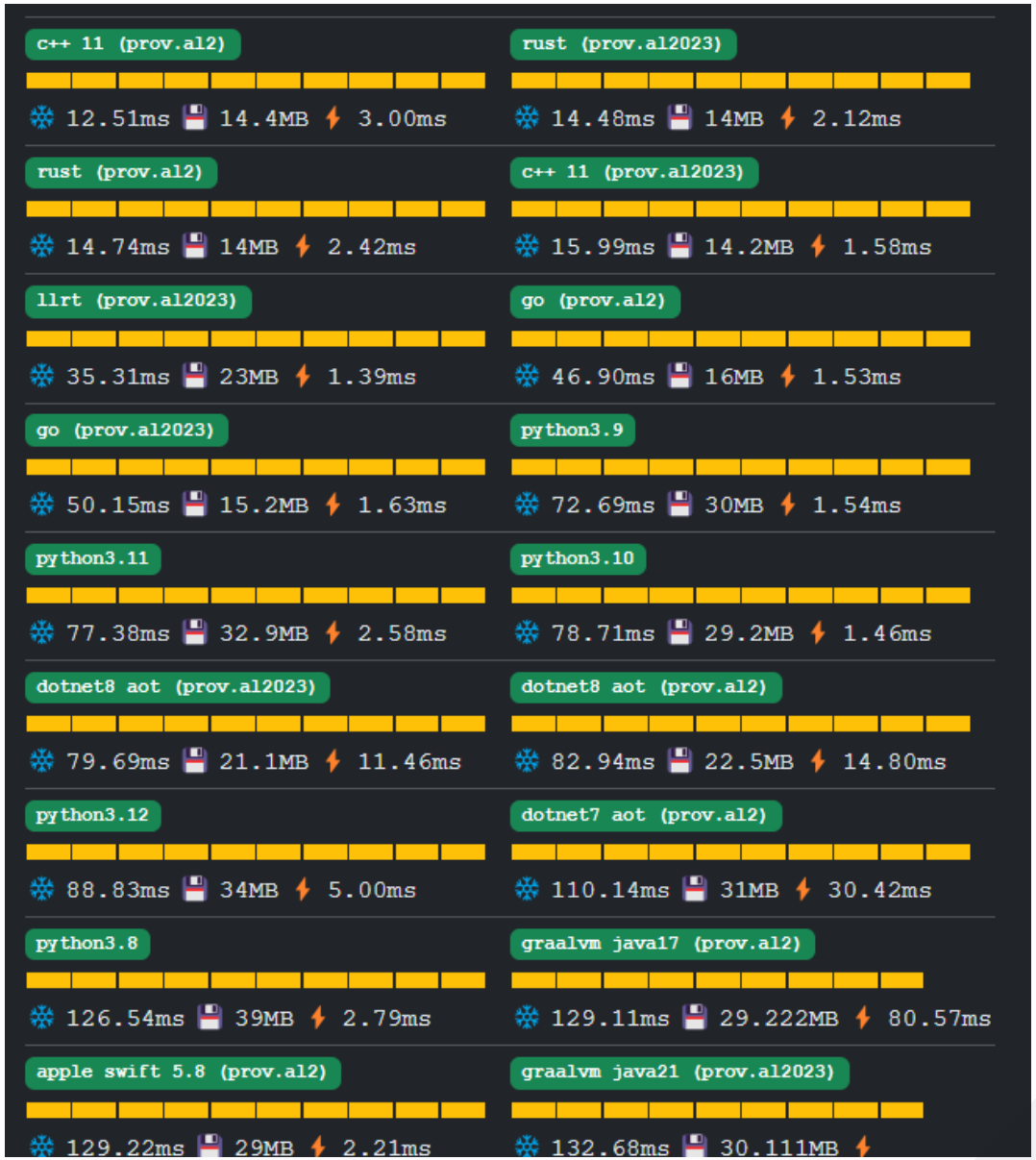
- Zed.dev code editor. *(Mac Only)*
- Project Oxidation at Mozilla[https://wiki.mozilla.org/Oxidation]

AWS Support

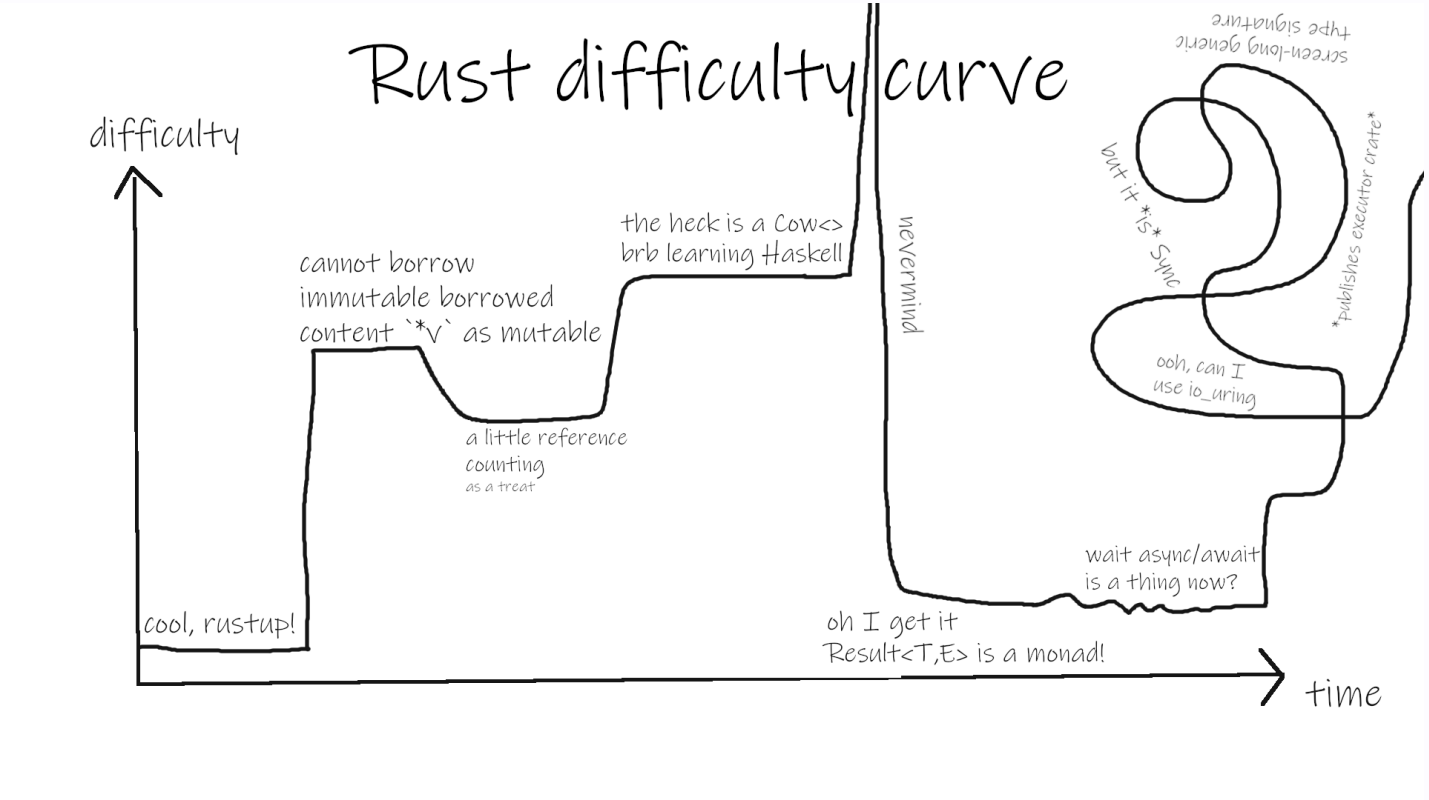
- AWS Lambda supports Amazon Linux 2023
- Cargo Lambda and docs
- Serverless Rust

AWS Lambda Cold Starts

Lambda Cold Starts Benchmark



Why not Rust?



Large cognitive load to learn.

- Limits the creative flow with it's constraints. No driving your car off the cliff for fun.
- Compiler is your friend and your foe. Lifetimes are difficult but the compiler will give you hints.

- Compile time is longer (than Go, etc)
- Not Popular - https://madnight.github.io/githut
- Many prefer Go (for webservers and quick prototyping)
- You want quick code to production and you are happy to have errors in production - Rust causes you to think about edge cases and errors earlier. **You pay early with Rust.**

Thinking Non-Functional

- Maintainability (Get it right early - not later)
- Testing (By design)
- Scalability (Low Memory)
- You can prototype with the code but it encourages you to have polished code early.

Learn

Great! - where do I get started?

Official https://www.rust-lang.org/learn

- The Book - The most comprehensive resource for learning Rust, but a bit theoretical sometimes. You will be using this along with Rustlings. Available in paperback and ebook format from No Starch Press..
- Rust Book (Interactive) - Different version of the Rust Book, featuring: quizzes, highlighting, visualizations, and more.
- Rust by Example - Learn Rust by solving little exercises. It's almost like rustlings, but online

Stay up to date with This Week in Rust - weekly newsletter and RSS Feed

Practise

- Rust Playground - Run rust without installing anything
- Rustlings - Contains small exercises to get you used to reading and writing Rust code
- Exercism - Rust Track - Series of exercises to learn rust
- And more...

```
---
title: Learn with Rustlings
---
stateDiagram-v2
    [*] --> Compile
    Compile --> Fix
    Fix --> Compile
    Compile --> Next
    Next --> Compile
    Next --> [*]
```

Learn by Video (cont...)

- Rust In Motion
- Beginners Series To Rust
- Rust First Steps
- How to Learn Rust

Dev Container

Rust might be flagged by anti-virus software - develop in a container instead 😏

devcontainer.json

```
{
    "name": "Rust",
    "image": "mcr.microsoft.com/devcontainers/rust:latest",
    "customizations": {
        "vscode": {
            "settings": {
                "editor.defaultFormatter": "rust-lang.rust-analyzer",
                "editor.formatOnSave": true,
                "explorer.fileNesting.enabled": true,
                "explorer.fileNesting.expand": false,
                "explorer.fileNesting.patterns": {
                    "Cargo.toml": ".clippy.toml, .rustfmt.toml, Cargo.lock, clippy.toml, cross.toml, rust-toolchain.toml, rustfmt.toml"
                },
                "todo-tree.regex.regex": "(//|#|<!--|;|/\\*|^|^[ \\t]*(-|\\d+.))\\s*($TAGS)|todo!",
            },
            "extensions": [
                "dustypomerleau.rust-syntax",
                "serayuzgur.crates",
                "Gruntfuggly.todo-tree",
                "streetsidesoftware.code-spell-checker"
                "tamasfe.even-better-toml",
                "usernamehw.errorlens",
                "vadimcn.vscode-lldb",
            ]
        }
    }
}
```

Tools

- Cargo - use this - don't use `rustc`
- RustFmt - standardize formatter
- Clippy - better code feedback than the compiler

Editions

2021 is the current version - Install and upgrade with https://rustup.rs/

Every three years, 2015, 2018, 2021, 2024. Defaults to 2015 if you don't specify otherwise. Can have breaking changes. You can choose the edition in the Cargo.toml and choose when to upgrade. They provide tooling to automate the upgrade. See more

Hello World

```
$ cargo new hello_world
     Created binary (application) `hello_world` package
$ cd hello_world/
$ cargo run
   Compiling hello_world v0.1.0 (/workspaces/rust/hello_world)
    Finished dev [unoptimized + debuginfo] target(s) in 1.27s
     Running `target/debug/hello_world`
Hello, world!
```

```
$ tree
.
├── Cargo.lock
├── Cargo.toml
├── src
│   └── main.rs
└── target
    └── debug
        └── hello_world //native executable

10 directories, 18 files (redacted)
```

```
real    0m0.018s
user    0m0.003s
sys     0m0.001s
```

```
$ time go build

real    0m0.041s
user    0m0.055s
sys     0m0.068s


$ time go run hello.go
hello world

real    0m0.079s
user    0m0.148s
sys     0m0.017s
```

```
$ time python helloworld.py
Hello, World!
```

```rust
//main.rs
fn main() {
    println!("Hello World");
}
```

Features

Variables - immutable (cannot be changed) by default

```
let x = 4;        // Compiler decides
let x: i32 = 4; // Explicit Definition
let x = 4i32;    // Suffix Annotations
```

```rust
let mut x = 5;
println!("x = {x}");
x = 6;
println!("x = {x}");
```

Strings

```
//Create strings
let _x = String::from("string");
let _y = "string".to_string();

//Use strings
let text: &str = "Hello\nworld";
let upper: String = text.to_uppercase();
let stripped = upper.strip_prefix("HELLO\n").unwrap();
println!("{}", first_line(stripped));

//String slices                    //0123456789X1234567
let sentence: String = String::from("This is a sentence");
let string_slice: &str = &sentence[..4];
println!("'{}'", string_slice);
let string_slice: &str = &sentence[5..9];
println!("'{}'", string_slice);
let string_slice: &str = &sentence[10..];
println!("'{}'", string_slice);

//these two are the same
let string_slice: &str = &sentence[..];
println!("'{}'", string_slice);
let string_slice: &str = &sentence;
println!("'{}'", string_slice);
```

Scalars

```
let a: i32 = 5;
    let b: u8 = 1;
    let x: f64 = 2.0;
    let y: f32 = 3.0;

    //prefix with underscore to remove 'unused variable' warning
    let _sum = a + b as i32;
    let _difference = x - 1.0;
    let _product = 4 * a;
    let _quotient = 9.0 / y;
    let _remainder = a % b as i32;

    let _t: bool = true;
    let _f: bool = false;

    let _c: char = '😍';
    let _d: char = 'a';

    //arrays
    let _my_array: [i32; 3] = [1, 2, 3];
    let _first = _my_array[0];

    //tuples

    //empty tuples are called 'unit'
    let unit = (); //empty value or an empty return type.

    let _tuple: (bool, i32, f64) = (true, 2, 3.0);
    let (b, i, f) = _tuple; //destructuring
    println!("{b}, {i}, {f}");

    let b: bool = _tuple.0;
    let i: i32 = _tuple.1;
    let f: f64 = _tuple.2;
    println!("{b}, {i}, {f}");
```

Functions

```
fn exclaim(input: String) -> String {
    let mut output = input.to_uppercase();
    output.push('!');
    output //same as "return output;"
}
```

Loops

```
//loop
    let mut i: i32 = 0;
    const MAX_LOOP: i32 = 5;
    loop {
        println!("I loop forever {}", i);
        i += 1;
        if i > MAX_LOOP {
            break;
        }
    }

    //assigning value from loop
    let an_array: [u32; 5] = [1, 2, 3, 4, 5];
    let mut i: usize = 0;
    let mut sum: u32 = 0;
    let result: u32 = loop {
        sum += an_array[i] * an_array[i + 1];
        i += 1;
        if i >= an_array.len() - 1 {
            break sum;
        }
    };
    println!("Assigned from loop value {}", result);

    //loop labels
    'loop_outer: loop {
        loop {
            println!("Inner loop run once");
            break 'loop_outer;
        }
        //println!("Never Reached");
    }

    //while
    let mut number = 3;
    while number != 0 {
        println!("{}", number);
        number -= 1;
    }
    println!("Lift Off!");

    //for
    let a: [i32; 5] = [10, 20, 30, 40, 50];

    for element in a.iter() {
        println!("the value is: {}", element);
    }

    //for example

    let x: f64 = 2_f64.sqrt().powf(3.0);
    println!("{}", x);

    //demo
    let mut i: i32 = 1;
    let something: i32 = loop {
        i *= 2;
        if i > 100 {
            break i;
        }
    };
    assert_eq!(something, 128);

    let mut counter: i32 = 0;
    while counter < 10 {
        print!("{}", counter);
        counter += 1;
    }
    println!();

    for number in (0..5).rev() {
        print!("{number}...")
    }
    println!("Lift off!");
```

Enums

```
enum CardinalDirections {
    North,
    South,
    East,
    West,
}

let north = CardinalDirections::North;
```

```
//option enum
enum Option<T> {
    Some(T),
    None,
}
```

```
    model: String,
    year: u32,
}

fn print_car(car: &Car) {
    println!(
        "Car is {} {} and was built in {}",
        car.make, car.model, car.year
    );
}
```

```
let car1 = Car {
    make: String::from("Ford"),
    model: String::from("Mustang"),
    year: 1967,
};

print_car(&car1);
```

```
println!("{:?}", car1); //Single line - good for small
```

Implementing Structs

```rust
struct Film {
    title: String,
    director: String,
    studio: String,
}

struct Book {
    title: String,
    author: String,
    publisher: String,
}

struct Album {
    title: String,
    artist: String,
    label: String,
}

trait Catalog {
    fn describe(&self) {
        println!("We need more information about this type of media")
    }
}

impl Catalog for Film {
    fn describe(&self) {
        println!(
            "{} was directed by {} through {} studio",
            self.title, self.director, self.studio
        );
    }
}

impl Catalog for Book {
    fn describe(&self) {
        println!(
            "{} was written by {} and published by {} studio",
            self.title, self.author, self.publisher
        );
    }
}

impl Catalog for Album {
    fn describe(&self) {
        println!(
            "{} is sung by {} and their record label is {}",
            self.title, self.artist, self.label
        );
    }
}

fn main() {
    let captain_marvel = Film {
        title: String::from("Captain Marvel"),
        director: String::from("Anna Boden and Ryan Fleck"),
        studio: String::from("Marvel"),
    };

    captain_marvel.describe();

    let elantris = Book {
        title: String::from("Elantris"),
        author: String::from("Brandon Sanderson"),
        publisher: String::from("Tor Books"),
    };

    elantris.describe();

    let let_it_be = Album {
        title: String::from("Let it Be"),
        artist: String::from("Beatles"),
        label: String::from("Apple"),
    };

    let_it_be.describe();
}
```

Error Handling

```
panic!("in the disco"); //🪩💃
```

```
// #1
let f = File::open("hello.txt");
let f = match f {
    Ok(file) => file,
    Err(error) => panic!("Can't opent the file: {:?}", error),
};

// #2
let f = File::open("hello.txt").unwrap(); //default error message of 'No such file or directory'.

// #3
let f = File::open("hello.txt").expect("Failed to open hello.txt");
```

// Out of bounds

```
let v: Vec<i32> = vec![0, 1, 2, 3];
println!("{}", v[6]);
```

Ownership

Rules of Ownership

- Each value in Rust has a variable that is called it's *owner*
- There can only be *one owner* at a time
- When the owner goes *out of scope*, the *value* will be dropped.

```
fn main() {
    let mut say: String = String::from("Ca");
    say.push('t');
    let say2 = say.clone(); //with "let say2 = say;' say2 is new owner. Invalidates say as original variable. Clone allocates a new pointer on the stack and copies the data on the heap.

    print_out(&say2); //Borrowing
                      //println!(say); //Error
    drop(say); //okay to drop say
    println!("Again: {}", say2);

    //Borrowing with a mutable reference
    let mut my_vec: Vec<i32> = vec![1, 2, 3];
    println!("{:?}", my_vec);
    add_to_vec(&mut my_vec);
    println!("{:?}", my_vec);
}

fn print_out(to_print: &String) {
    println!("{}", to_print);
}

fn add_to_vec(a_vec: &mut Vec<i32>) {
    a_vec.push(4);
}
```

```
assert_eq!(pizza.inches, 16);
}
```

```
## Document Tests
```

```
/// This function divides two numbers.
///
/// # Example #1: 10 / 2 == 5
///
/// /// let result = document_tests::div(10,2);
/// assert_eq!(result, 5); ///
///
```

- Ownership is more than memory
  - socket management (GC doesn't release unused sockets)
  - Mutex, RC, File, our own struts that implement `Drop` trait - `drop(&mut self)

- Borrowing is the most idiomatic solution
  - Avoids cloning as that uses extra memory and time unnecessarily
  - Catches issues at compile time
    - C: Segment on fault
    - JavaScript: Undefined is not a function

Idiomatic Solution Example

```
result.is_some ❌

match result { 👌 }

result? ✅
```

- Strings
  - String object data (points to actual UTF-8 data),length,capacity
  - Slices: Borrow checker always ensures slices are aways valid at compile time
- Compile time errors

```rust
let mut shopping_list: Vec<&str> = Vec::new();
//Vec<&str> not needed as the compiler can infer it from the next line
shopping_list.push("milk");
```

- Lifetimes
- Return 'unit' (empty tuple)
- Return Result and match //no try catch
- Optional Results

```rust
pub enum Option<T> {
    None,
    Some(T),
} //leads to unwrap
```

Error Handling

```
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

Rather than having at catch and only catching specific exceptions. Return a custom error enum and the caller to your function has to handle all types of errors.

- Enums and Traits. Polymorphism by default. Traits are like interfaces.
  Forces early abstraction - can be good and bad.

- String/Array Slices (no disconnect from the index.)
  - A slice is a view into sequence of elements. A fat pointer contains the memory location and the length of the data. Opposed to having a zero '/0' terminated string.
    Slices - Reference to a contiguous subset of data in another data structure.
- Unit Testing inbuilt
- Macro expansion
- SQL is actually run at compile time in a transaction to ensure that the SQL is valid and then rolled back

- Calling rust code from other languages
  - https://github.com/PyO3/pyo3
- Tests

Controversy

- Leadership issues
- Trademark Policies
- Choice of Malware writers - 'RustDoor'